



Collider PixelCollions,
Lazy, Eager, State





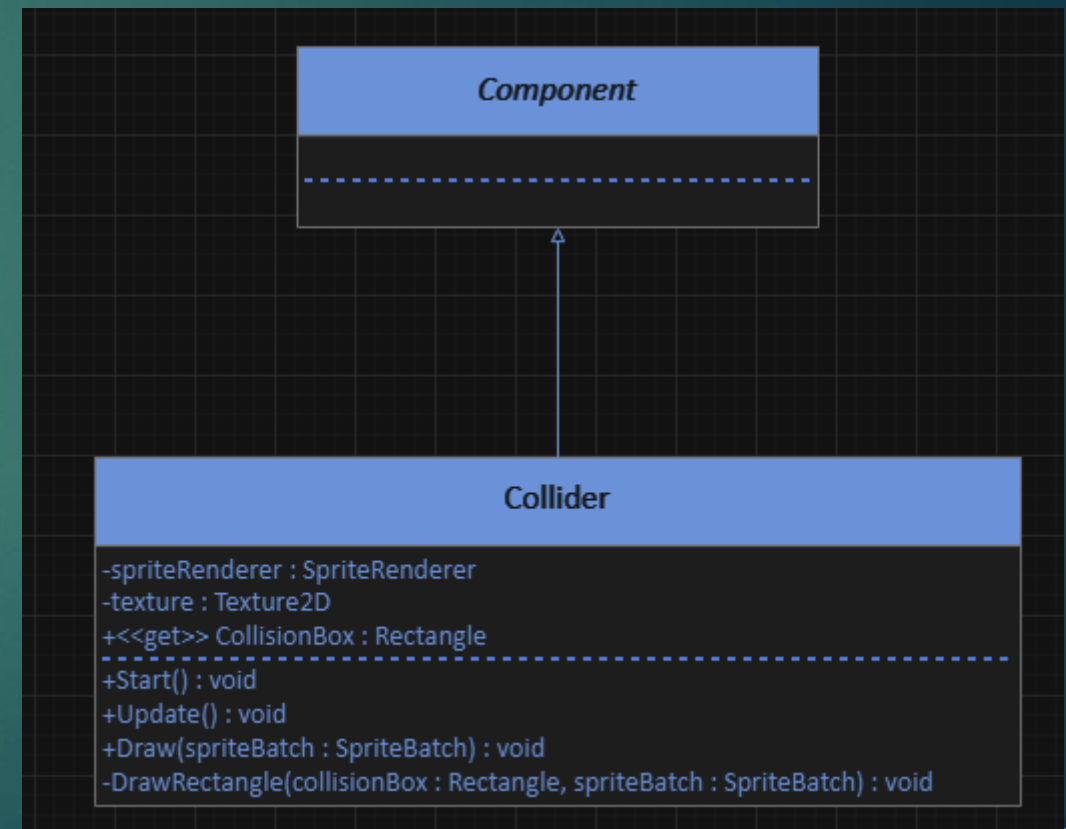
Undervisning
https://www.youtube.com/watch?v=162_yyJnRGY

COLLIDER



COLLIDER

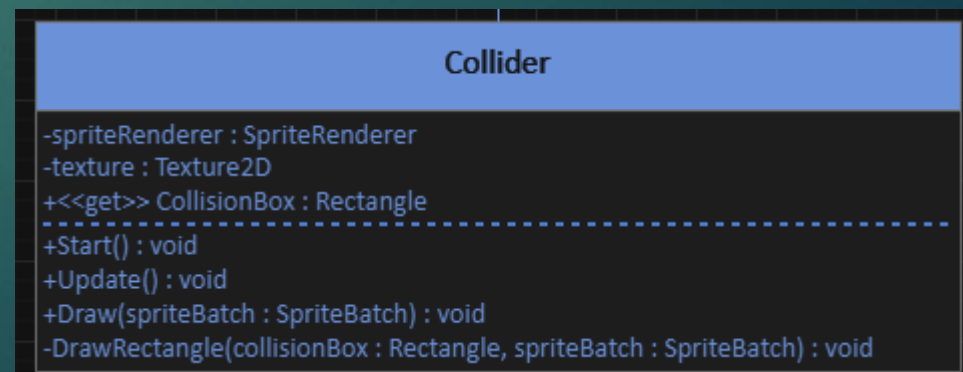
- ▶ Vi skal oprette en Collider component



COLLIDER

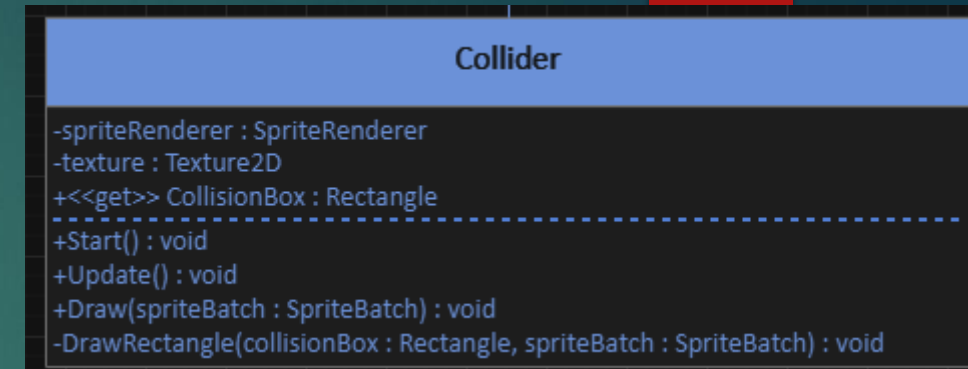
- ▶ Vi skal oprette en Collider component
 - ▶ CollisionBox
 - ▶ Obs: SpriteRenderer reference skal sættes i start!

```
public Rectangle CollisionBox
{
    get
    {
        return new Rectangle
        (
            (int)(GameObject.Transform.Position.X - spriteRenderer.Sprite.Width / 2),
            (int)(GameObject.Transform.Position.Y - spriteRenderer.Sprite.Height / 2),
            spriteRenderer.Sprite.Width,
            spriteRenderer.Sprite.Height
        );
    }
}
```



COLLIDER

- ▶ Vi skal oprette en Collider component
 - ▶ DrawRectangle (kaldes af Draw)
 - ▶ OBS: "texture" er blot en enkelt pixel der er loaded ind!
 - ▶ Loades ind i start.



```
private void DrawRectangle(Rectangle collisionBox, SpriteBatch spriteBatch)
{
    Rectangle topLine = new Rectangle(collisionBox.X, collisionBox.Y, collisionBox.Width, 1);
    Rectangle bottomLine = new Rectangle(collisionBox.X, collisionBox.Y + collisionBox.Height,
        collisionBox.Width, 1);
    Rectangle rightLine = new Rectangle(collisionBox.X + collisionBox.Width, collisionBox.Y, 1,
        collisionBox.Height);
    Rectangle leftLine = new Rectangle(collisionBox.X, collisionBox.Y, 1, collisionBox.Height);

    spriteBatch.Draw(texture, topLine, null, Color.Red, 0, Vector2.Zero, SpriteEffects.None, 1);
    spriteBatch.Draw(texture, bottomLine, null, Color.Red, 0, Vector2.Zero, SpriteEffects.None, 1);
    spriteBatch.Draw(texture, rightLine, null, Color.Red, 0, Vector2.Zero, SpriteEffects.None, 1);
    spriteBatch.Draw(texture, leftLine, null, Color.Red, 0, Vector2.Zero, SpriteEffects.None, 1);
}
```





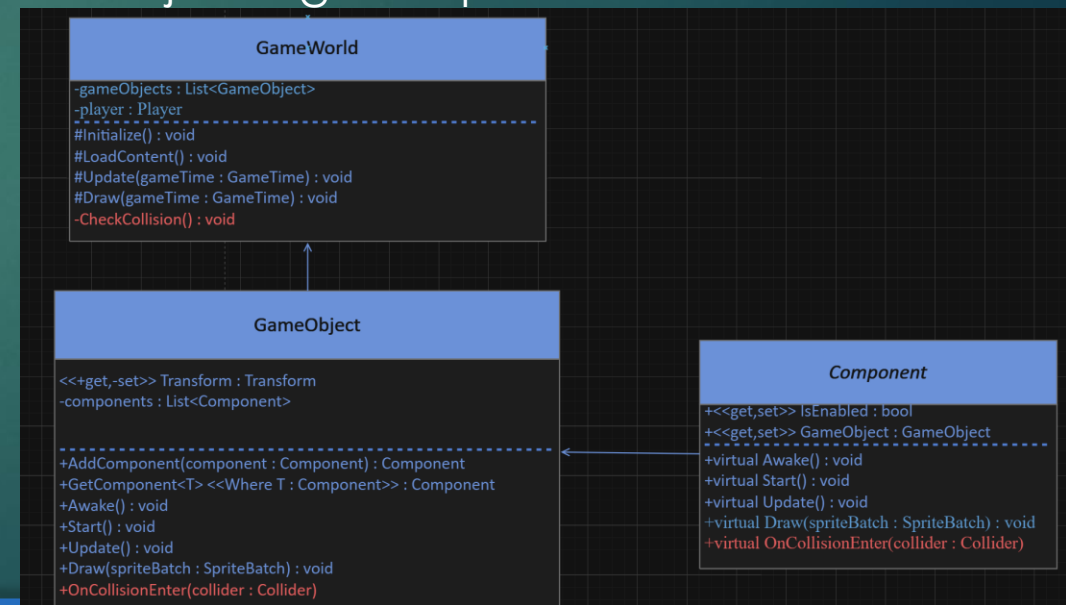
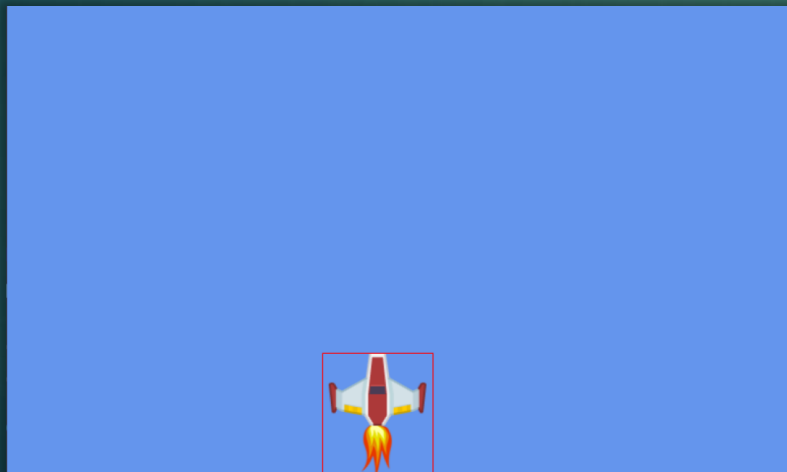
COLLISION



COLLISION

► Collision:

- Vi har nu kasser på vores objekter
- Det skal være muligt at finde ud af om 2 colliders kolliderer, og give beskeden videre til interesserede komponenter.
- Dette kræver en udvidelse i gameworld, GameObject og Component.



COLLISION

► CheckCollision()

- Løber gameObjekter i update, og finder alle colliders, og på alle disse tjekker om de collider.
- Såfremt der er en collision bliver OnCollisionEnter kaldt på begge gameObjekter

```
void CheckCollision()
{
    foreach (GameObject go1 in gameObjects)
    {
        foreach (GameObject go2 in gameObjects)
        {
            if (go1 == go2)
            {
                continue;
            }
            Collider col1 = go1.GetComponent<Collider>() as Collider;
            Collider col2 = go2.GetComponent<Collider>() as Collider;

            if (col1 != null && col2 != null && col1.CollisionBox.Intersects(col2.CollisionBox))
            {
                go1.OnCollisionEnter(col2);
                go2.OnCollisionEnter(col1);
            }
        }
    }
}
```

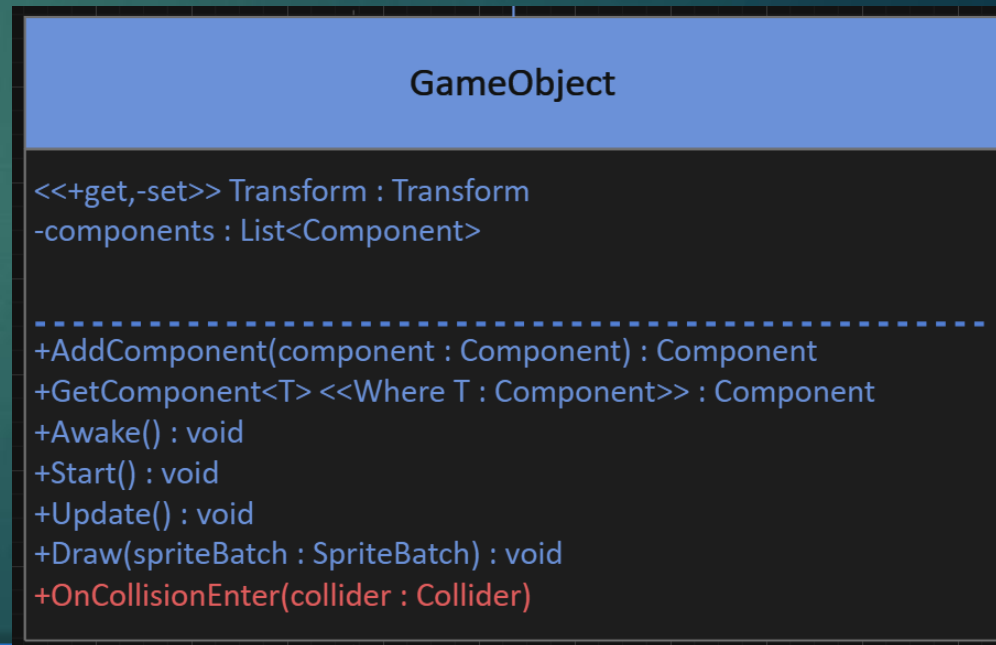
GameWorld

```
-gameObjects : List<GameObject>
-player : Player
-----
#Initialize() : void
#LoadContent() : void
#Update(gameTime : gameTime) : void
#Draw(gameTime : gameTime) : void
-CheckCollision() : void
```


COLLISION

- ▶ OnCollisionEnter(Collider Collider)
 - ▶ Sender blot informationen videre til alle dens components
 - ▶ Præcis ligesom draw, update, etc.

```
2 references
public void OnCollisionEnter(Collider collider)
{
    for (int i = 0; i < components.Count; i++)
    {
        components[i].OnCollisionEnter(collider);
    }
}
1 reference
```



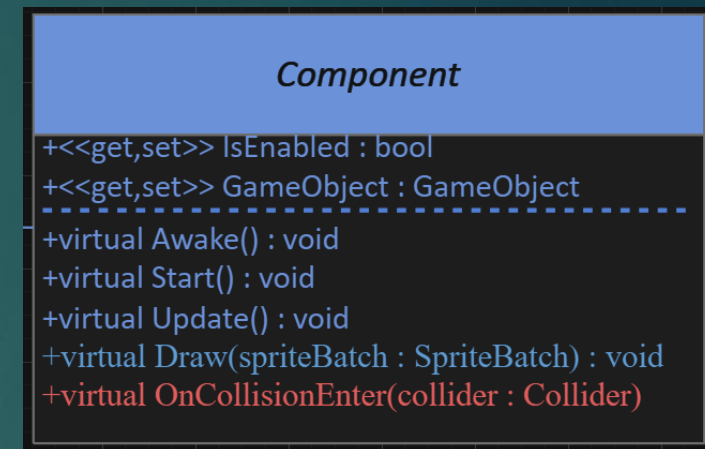
COLLISION

- ▶ Brugen af OnCollisionEnter kunne f.eks være således:

```
public override void OnCollisionEnter(Collider col)
{
    if (col.GameObject.GetComponent<Player>() != null)
    {
    }
}
```

- ▶ Dette er på en component "Enemy"

- ▶ Både Enemy og Player gameobjekterne har en Collider component.
- ▶ Eventet her bliver derfor kaldt når en enemy collider med en Player
- ▶ Husk at kun implementer kode en gang!
 - ▶ Collisions eventet bliver jo kaldt på både enemy og Player.

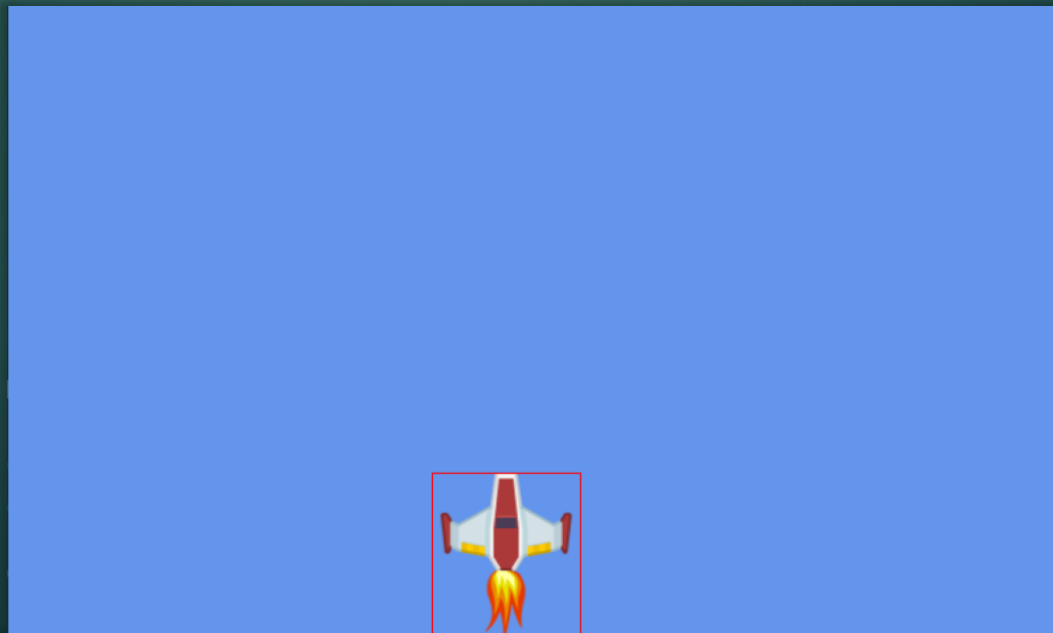


ØVELSE 16

Løsning

<https://www.youtube.com/watch?v=gNaSE3u3lds>

- ▶ Implementer et Collider component og tilføj det til spilleren.
 - ▶ Sørg for at collideren bliver tegnet.
 - ▶ Sørg for at det er muligt at slå draw'ing af colliderers til og fra (på alle gameobjects) fra ved at trykke på en knap



ØVELSE 17

Løsning
https://www.youtube.com/watch?v=TniGI5Mw_BE

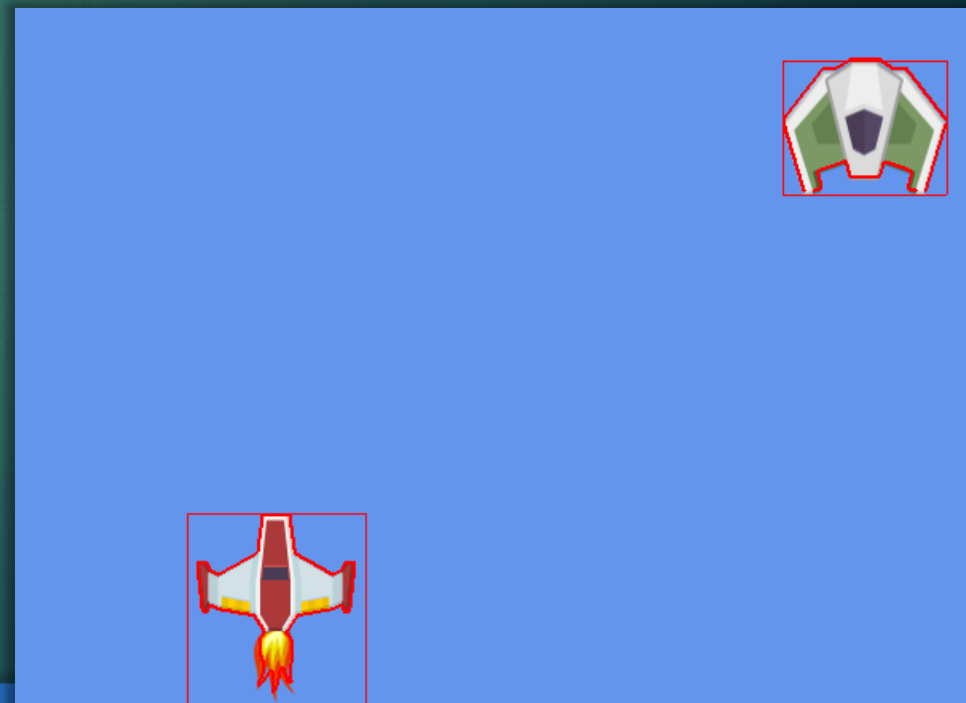
- ▶ Tilføj en collider til din enemy.
- ▶ Tilføj collision kode til din GameWorld, GameObject og Component
- ▶ Sørg for at dine enemies bliver fjernet fra spillet, hvis de kolliderer med din player.
 - ▶ Husk at du stadig skal benytte dig af din objectpool.
 - ▶ Dvs. at døde enemies skal tilbage i din objekt pool
 - ▶ Husk at fjerne colliders fra inaktive Enemies fra din GameWorld.
 - ▶ Husk at kun invoke fjernelse fra enten spiller enemy componenten.

Undervisning
<https://www.youtube.com/watch?v=YlizAm6mqAQ>

PIXEL COLLISION

PIXEL COLLISION

- ▶ Vi skal lave en pixel collider
 - ▶ Selvom vi bruger en PixelPerfect collider, så skal vi stadig bruge vores BoxCollider.
 - ▶ En pixel collider består af mange små rectangles rundt kanten af en sprite.
 - ▶ De er dyre i drift, så vi bruger en hybrid



PIXEL COLLISION

1. Det første vi skal gøre er at få fat på alle Pixels.
 - ▶ Dette gøres i metoden CreateRectangles.
 - ▶ For at få fat på alle pixels kan man bruge metoden GetData på en Texture2D.
 - ▶ I nedenstående eksempel smides hver linje af pixels på spriten ind i et color array.

```
List<Color[]> lines = new List<Color[]>();

for (int y = 0; y < spriteRenderer.Sprite.Height; y++)
{
    Color[] colors = new Color[spriteRenderer.Sprite.Width];
    spriteRenderer.Sprite.GetData(0, new Rectangle(0, y,
        spriteRenderer.Sprite.Width, 1), colors, 0,
        spriteRenderer.Sprite.Width);
    lines.Add(colors);
}
```


PIXEL COLLISION

2. Vi har brug for at lave vores egen RectangleData klasse, til at holde styr på hver enkelt rectangle i vores PixelCollider.
 - X & Y er positionen er den specifikke Pixel denne Rectangle skal placeres på. (f.eks. 10, 10).

```
public class RectangleData
{
    5 references
    public Rectangle Rectangle { get; set; }

    2 references
    public int X { get; set; }

    2 references
    public int Y { get; set; }

    1 reference
    public RectangleData(int x, int y)
    {
        this.Rectangle = new Rectangle();
        this.X = x;
        this.Y = y;
    }

    1 reference
    public void UpdatePosition(GameObject gameObject, int width, int height)
    {
        Rectangle = new Rectangle((int)gameObject.Transform.Position.X + X - width / 2, (int)
        gameObject.Transform.Position.Y + Y - height / 2, 1, 1);
    }
}
```

RectangleData

```
+<<get,set>> Rectangle : Rectangle
+<<get, set>> X : int
+<<get, set>> Y : int
-----
+RectangleData(x : int, y : int)
+UpdatePosition(gameObject : GameObject, width : int, height : int)
```


PIXEL COLLISION

3. Alle vores pixel linjer ligger nu i vores lines liste.

```
List<Color[]> lines = new List<Color[]>();
```

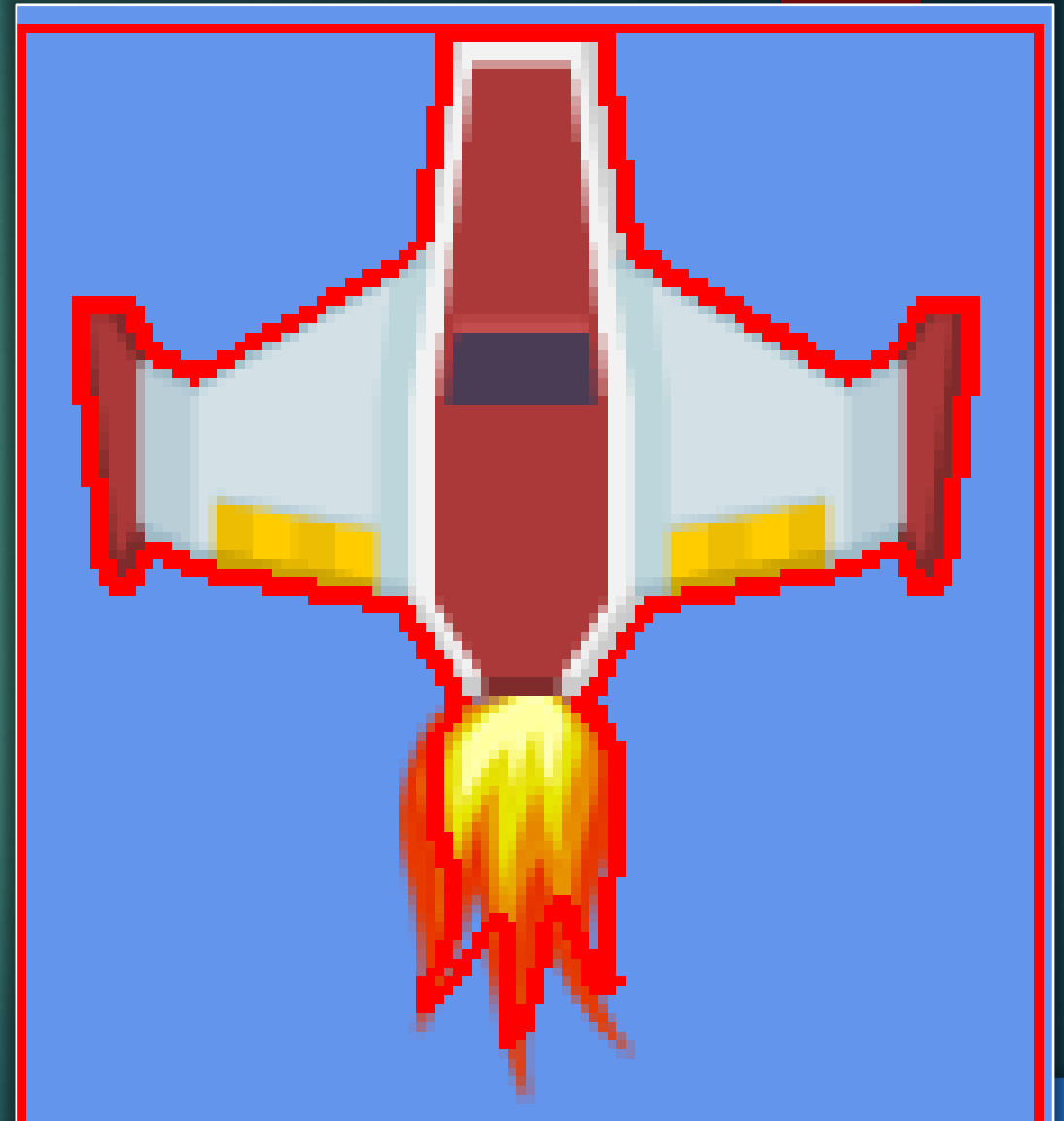
- ▶ Vi kan bruge disse linjer til at oprette vores pixel collider, baseret på følgende kriterier, gøres også i CreateRectangles:
 - ▶ Vi skal oprette en instans af RectangleData for hver pixel i kanten af spriten.
 - ▶ En pixel er i kanten af spriten, hvis en nabopixel har en alpha værdi på 0.
 - ▶ En pixel er i kanten af spriten, hvis alpha værdien ikke er 0 og den ligger i den første eller sidste linje på x eller y aksen.
 - ▶ Hver instans af RectangleData tilføjes til rectangles listen.

Collider

```
-spriteRenderer : SpriteRenderer  
-texture : Texture2D  
+<<get>> CollisionBox : Rectangle  
-rectangles : List<RectangleData>  
-----  
+Start() : void  
+Update() : void  
+Draw(spriteBatch : SpriteBatch) : void  
-DrawRectangle(collisionBox : CollisionBoc, spriteBatch : SpriteBatch) : void  
-CreateRectangles() : void  
-UpdatePixelCollider() : void
```

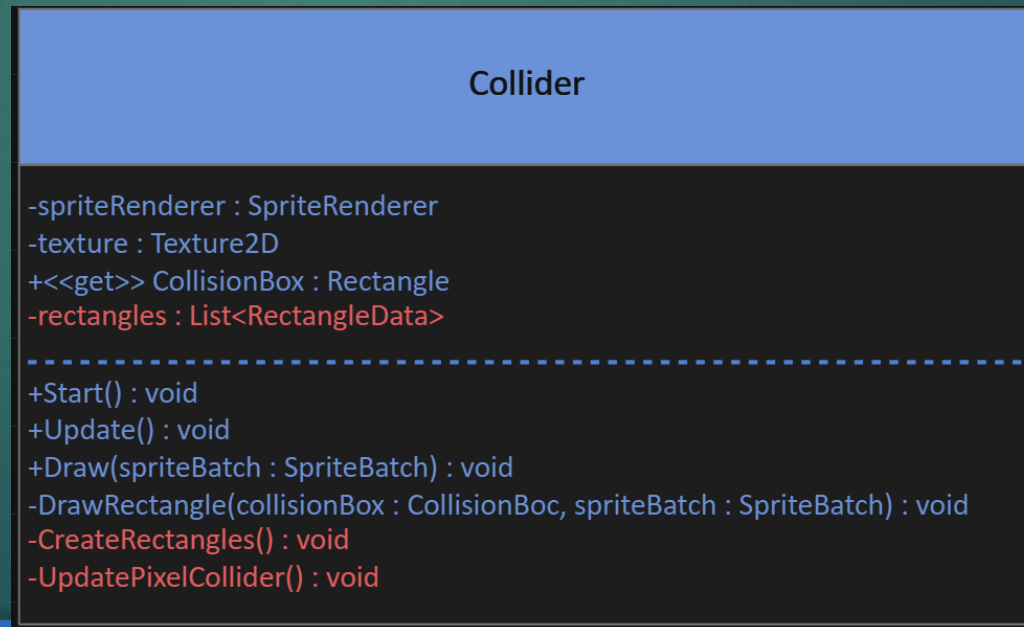
PIXEL COLLISION

```
for (int y = 0; y < lines.Count; y++)  
{  
    for (int x = 0; x < lines[y].Length; x++)  
    {  
        if (lines[y][x].A != 0)  
        {  
            if ((x == 0)  
                || (x == lines[y].Length)  
                || (x > 0 && lines[y][x - 1].A == 0)  
                || (x < lines[y].Length - 1 && lines[y][x + 1].A == 0)  
                || (y == 0)  
                || (y > 0 && lines[y - 1][x].A == 0)  
                || (y < lines.Count - 1 && lines[y + 1][x].A == 0))  
            {  
                RectangleData rd = new RectangleData(x, y);  
                rectangles.Add(rd);  
            }  
        }  
    }  
}
```



PIXEL COLLISION

4. UpdatePixelCollider kalder UpdatePosition på alle RectangleData objekter.
 - ▶ Hvis metoden ikke kaldes bliver collideren ikke flyttet med spilleren.



ØVELSE 18

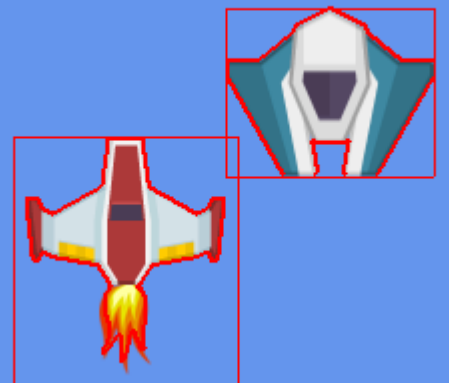
Løsning
<https://www.youtube.com/watch?v=Vw2jJ5ufWHI>

- ▶ Tilføj PixelCollision til din spillere og Enemies.
 - ▶ Der må ikke checkes for PixelCollision med mindre der allerede er sket en BoxCollision.
 - ▶ Enemies skal kun fjernes hvis de rammer din player med en PixelCollision.

**CHECK FOR
BOX COLLISION**



**CHECK FOR
PIXEL COLLISION**



Lazy loading

Undervisning
<https://www.youtube.com/watch?v=9GGuXoMH6vo>

► Formål

- Lazy loading er en pattern/koncept der udskyder loading af objekter indtil de skal bruges
- Det modsatte af lazy loading er eager loading, alt loades ind i hukommelsen, så det er klar til brug.

Lazy loading

► Fordele

- Kan gøre opstartstiden for applikationen kortere
- Applikationen bruger generelt mindre hukommelse pga. "on demand loading"

► Ulemper

- Kan gøre koden kompliceret da vi skal kontrollere om det er nødvendigt at loade vores objekter

Lazy loading

► Eksempel:

► Eager

```
Armory myArmory = new Armory();  
  
//Open armory  
foreach (Weapon wpn in myArmory.Weapons)  
{  
    Console.WriteLine(wpn.Type);  
}
```

Name	Value
args	{string[0]}
myArmory	{LazyLoading.Armory}
Weapons	Count = 3
weapons	Count = 3

```
class Armory  
{  
    private List<Weapon> weapons;  
  
    public List<Weapon> Weapons{...}  
  
    public Armory()  
    {  
        LoadWeapons(); //Loads the weapons right away  
    }  
  
    public void LoadWeapons()  
    {  
        weapons = new List<Weapon>()  
        {  
            new Weapon("AK47", 30),  
            new Weapon("Glock",10),  
            new Weapon("Granade",1)  
        };  
    }  
}
```


Lazy loading

► Eksempel:

► Lazy

```
Armory myArmory = new Armory();  
  
//Open armory  
foreach (Weapon wpn in myArmory.Weapons)  
{  
    Console.WriteLine(wpn.Type);  
}
```

Name	Value
► myArmory.weapons	null

```
public List<Weapon> Weapons  
{  
    get  
    {  
        if (weapons == null)  
        {  
            LoadWeapons();  
        }  
        return weapons;  
    }  
}  
  
public void LoadWeapons()  
{  
    weapons = new List<Weapon>()  
    {  
        new Weapon("AK47", 30),  
        new Weapon("Glock", 10),  
        new Weapon("Granade", 1)  
    };  
}
```


14. Lazy loading

- ▶ Lazy loading in .NET
- ▶ Lazy<T>

<http://msdn.microsoft.com/en-us/library/dd642331.aspx>

Name	Description
IsValueCreated	Gets a value that indicates whether a value has been created for this Lazy<T> instance.
Value	Gets the lazily initialized value of the current Lazy<T> instance.

Lazy loading

► Brug af Lazy<T>

```
private Lazy<List<Weapon>> weapons;

public List<Weapon> Weapons
{
    get
    {
        return weapons.Value;
    }
}

public Armory()
{
    weapons = new Lazy<List<Weapon>>(() => LoadWeapons());
}

public List<Weapon> LoadWeapons()
{
    List<Weapon> tmpWeapons = new List<Weapon>()
    {
        new Weapon("AK 47", 30),
        new Weapon("GLock", 10),
        new Weapon("Granade", 1)
    };

    return tmpWeapons;
}
```

Lazy loading

► Eksempel med et enkelt objekt

```
private Lazy<Weapon> myWeapon;  
  
public Weapon MyWeapon  
{  
    get { return myWeapon.Value; }  
}  
  
public Armory()  
{  
    myWeapon = new Lazy<Weapon>(() => new Weapon("Sniper",10));  
}
```

ØVELSE 19

Løsning

https://www.youtube.com/watch?v=3psZDgLF_x0&feature=youtu.be

- ▶ Implementer Lazy loading på jeres Pixel collider.
 - ▶ Pixel delen af jeres Collider skal først lodes når der sker en Box collision.

Undervisning
<https://www.youtube.com/watch?v=UQuAnFi6Xrl>

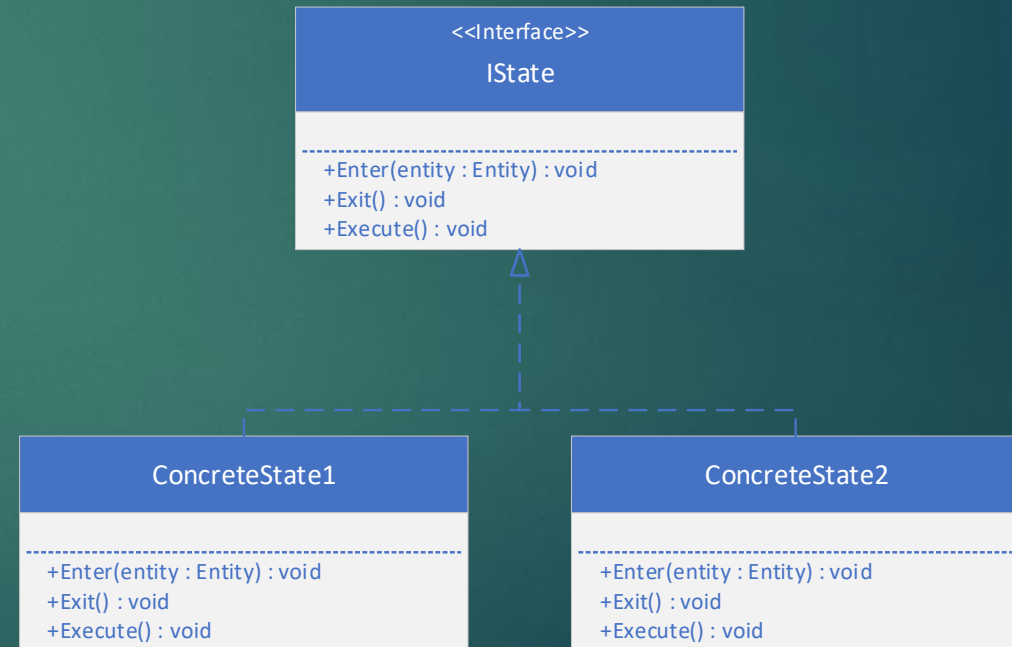
STATE PATTERN

State pattern

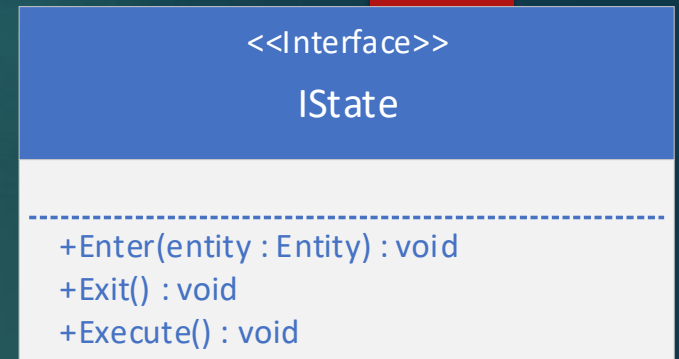
- ▶ Formål:
 - ▶ Bruges til at ændre et objekts adfærd under runtime.
 - ▶ Mere kontrol end strategy pattern.
- ▶ Bruges f.eks.
 - ▶ Bruges ofte til AI. Eller animation Til at skifte adfærd.
 - ▶ Idle → AI ser spilleren og skifter stadie
 - ▶ Walk → AI går efter spilleren til den er tæt nok på
 - ▶ Attack → Spilleren flytter sig væk
 - ▶ Walk → AI prøver at komme tæt på spilleren igen
 - ▶ Det er op til den individuelle state at skifte state!

State pattern

- ▶ Generel struktur på state pattern
 - ▶ Kan have flere metoder, efter behov.



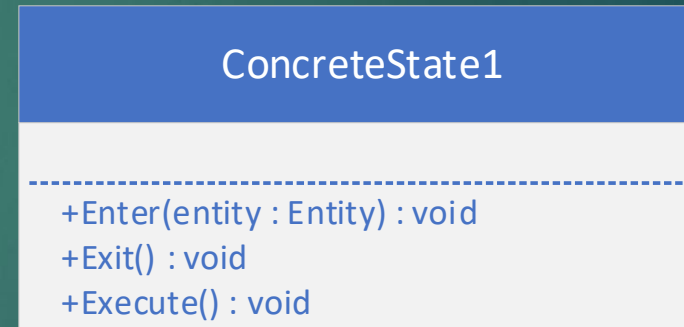
State pattern



- ▶ Istate et interface, som definerer hvad hvert state skal kunne.
 - ▶ Enter kaldes når der skiftes til et state. Dvs. Vi laver alle de opsætninger der skal til for at dette state kan fungerer.
 - ▶ Parametet entity er det objekt dette state kontrollerer.
 - ▶ F.eks. Walk → Run, har vi brug for at sætte speed til noget højere.
 - ▶ Vi skal have adgang til at lave om i disse fra **Entity**
 - ▶ Exit kaldes når vi forlader et state. Her kan vi rydde op hvis det er nødvendigt.
 - ▶ Har vi ændret i speed?
 - ▶ Sæt tilbage til hvad det var ved Enter
 - ▶ Execute kører hver frame så længe stadiet er aktivt.
 - ▶ Bestemmer om vi skal skifte til et andet state. F.eks. Er vi tæt nok på spilleren til at skifte til et attack state?

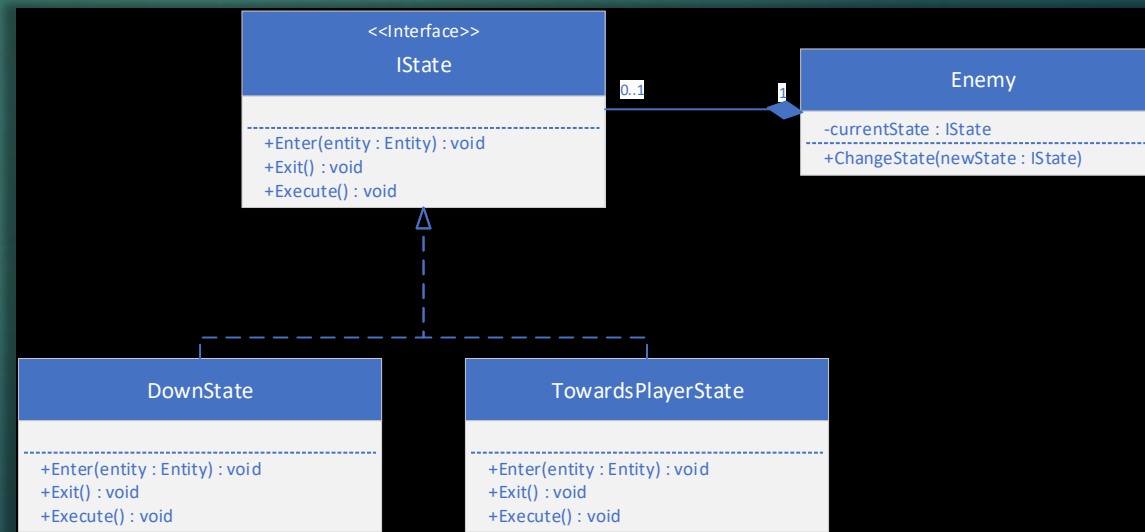
State pattern

- ▶ En konkret implementering af et state. Kan f.eks. Være idle, eller run



State pattern

- ▶ Et eksempel på en implementering i vores spil.
 - ▶ Enemy gør brug af et state til at udfører forskellige handlinger.
 - ▶ ChangeState bruges til at skifte til et nyt state på vores enemy.



Eksempel

- ▶ Interface struktur
 - ▶ Her er Enemy Entitien

```
public interface IState
{
    void Enter(Enemy parent);
    void Execute();
    void Exit();
}
```



- ▶ Er også muligt at sende Entity med til Execute og Exit, hvis der er behov
- ▶ Kunne også gøres generisk:

```
public interface IState<T>
{
    0 references
    void Enter(T parent);
    0 references
    void execute();
    0 references
    void Exit();
}
```

Eksempel

- ▶ Et state kan se ud på følgende måde

```
public class MoveState : IState<Enemy>
{
    private Enemy parrent;
    private Player player;
    float chaseDistance = 300f;
    2 references
    public void Enter(Enemy parrent)
    {
        this.parrent = parrent;
        player = GameWorld.Instance.Player;
    }

    2 references
    public void Execute()
    {
        parrent.MovementStrategy.Move();
        Vector2 playerPos = player.gameObject.Transform.Position;
        Vector2 enemyPos = parrent.gameObject.Transform.Position;

        if (Vector2.Distance(enemyPos, playerPos) < chaseDistance)
        {
            parrent.ChangeState(new RotateState());
        }
    }

    2 references
    public void Exit()
    {
    }
}
```

DownState

- +Enter(entity : Entity) : void
- +Exit() : void
- +Execute() : void

- ▶ OBS: Enemy's MovementStrategy er tilgængelig her via Property!
- ▶ Player er Property i GameWorld!

Eksempel

- ▶ Vi kan(og bør!) implementere en ChangeState metode på enemy, metoden bruges til at skifte stadie

```
public void ChangeState(IState newState)
{
    if (currentState != null)
    {
        currentState.Exit();
    }

    currentState = newState;
    currentState.Enter(this);
}
```

Enemy
-currentState : IState
+ChangeState(newState : IState)

Eksempel

- ▶ Brug af metoden i Enemy

```
public Enemy(GameObject gameObject, IMovementStrategy movementStrategy) : base(gameObject)
{
    this.movementStrategy = movementStrategy;
    currentState = new MoveState();
    ChangeState(currentState);
}
```

- ▶ Vi siger her at en spiller starter i MoveState når den bliver oprettet.

```
public override void Update()
{
    currentState.Execute();
}
```


ØVELSE 20

Løsning
https://www.youtube.com/watch?v=-aPA_UfnePw

- ▶ Implementer et state pattern på jeres enemies.
 - ▶ Der skal være 3 forskellige states:
 - ▶ MoveState
 - ▶ Bruger blot enemy'ens movement strategy og holder øje med når den er tæt på spilleren!
 - ▶ Sæt blot spiller til property i GameWorld.
 - ▶ RotateState
 - ▶ Roterer mod spilleren (mere på næste slide)
 - ▶ Skifter Enemy Sprite til rød
 - ▶ ChaseState
 - ▶ Bruger vores TopToBottomMovementStrategy, til at bevæge mod retningen af spilleren den har fået ved op start.
 - ▶ Husk at ryd op i de states hvor det er relevant, og når i sender enemies tilbage i poolen

Hints:

```
public void RotateTowards(Vector2 targetPosition)
{
    Vector2 direction = targetPosition - parrent.gameObject.Transform.Position;
    direction.Normalize();
    //make up for the fact that enemy sprites already are rotated!
    float desiredAngle = Mathf.Atan2(direction.Y, direction.X) - Mathf.PI/2;
    float angleDifference = desiredAngle - parrent.gameObject.Transform.Rotation;

    if (Math.Abs(angleDifference) < 0.01f)
    {
        parrent.gameObject.Transform.Rotation = desiredAngle;
        parrent.ChangeState(new ChaseState(direction));
        return;
    }
    float rotationStep = rotationSpeed * GameWorld.Instance.DeltaTime;

    if (angleDifference > 0)
    {
        parrent.gameObject.Transform.Rotation += Math.Min(rotationStep, angleDifference);
    }
    else if (angleDifference < 0)
    {
        parrent.gameObject.Transform.Rotation -= Math.Min(rotationStep, -angleDifference);
    }
}
```

Uddrag af RotateState.cs

Hints:

```
public override void Start()
{
    RespawnAndSetNewPosition();
}
1 reference
public void RespawnAndSetNewPosition()
{
    Random rnd = new Random();
    float xRandomCoord = rnd.Next(0, GameWorld.Instance.GraphicsDevice.Viewport.Width);
    float yCoordOutsideScreen = -((SpriteRenderer)gameObject.GetComponent<SpriteRenderer>()).Sprite.Height / 2;
    gameObject.Transform.Position = new Vector2(xRandomCoord, yCoordOutsideScreen);
}
```

- Ansvaret for at sætte position bør blive rykket tilbage i Enemy.cs

```
public interface IMovementStrategy
{
    6 references
    void Move();
}
```